Lab 2: Emotion Recognition from Twitter Text with DistilBERT

Yong Cuiyang
21307140051
School of Data Science
Fudan University

Abstract

This report presents an emotion recognition system for Twitter text using DistilBERT, a distilled variant of BERT. We compare two approaches: feature extraction, where DistilBERT serves as a fixed feature encoder and a softmax classifier is trained on the extracted [CLS] embeddings, and fine-tuning, where the entire model is adapted end-to-end via the HuggingFace Trainer API. Fine-tuning achieves approximately 94% validation accuracy, substantially outperforming feature extraction (approximately 63%). A $3 \times 3 \times 3$ grid search over learning rate, batch size, and weight decay is conducted in parallel across multiple GPUs to study their effects on fine-tuning performance. Error analysis reveals that class imbalance and vocabulary overlap are the primary causes of misclassification, particularly between the "surprise" and "joy" categories. Finally, we construct five adversarial examples through both manual crafting and a HotFlip gradient-based attack to probe the model's robustness. The source code and model are publicly available at `https://github.com/Snivallus/Introduction-to-AI-Project-02` and `https://huggingface.co/SnivellusSnape/distilbert-emotion`.

1 Introduction

Emotion recognition from text is a fundamental task in natural language processing with applications in social media monitoring, customer feedback analysis, and mental health assessment. Unlike traditional sentiment analysis which typically distinguishes only positive, negative, and neutral polarities, emotion recognition aims to identify fine-grained emotional states such as joy, sadness, anger, and fear.

The CARER dataset [Saravia et al., 2018] provides a suitable benchmark for this task: it contains approximately 20 000 English Twitter posts annotated with six basic emotions: sadness, joy, love, anger, fear, and surprise. The dataset exhibits substantial class imbalance, with "joy" and "sadness" being over-represented relative to "love" and "surprise."

Pre-trained Transformer models have become the dominant approach for text classification since the introduction of BERT [Devlin et al., 2019]. DistilBERT [Sanh et al., 2019], a distilled version of BERT that retains 97% of its performance while being 40% smaller and 60% faster, offers an attractive trade-off between accuracy and computational cost.

In this report, we fine-tune DistilBERT on the CARER emotion dataset using the HuggingFace Transformers library [Tunstall et al., 2022]. We investigate two paradigms: (i) feature extraction, where DistilBERT's weights are frozen and a softmax classifier is trained on the extracted [CLS] hidden states, and (ii) fine-tuning, where the entire model is updated end-to-end. We compare several Scikit-Learn classifiers within the feature extraction framework, conduct a hyperparameter

Preprint.

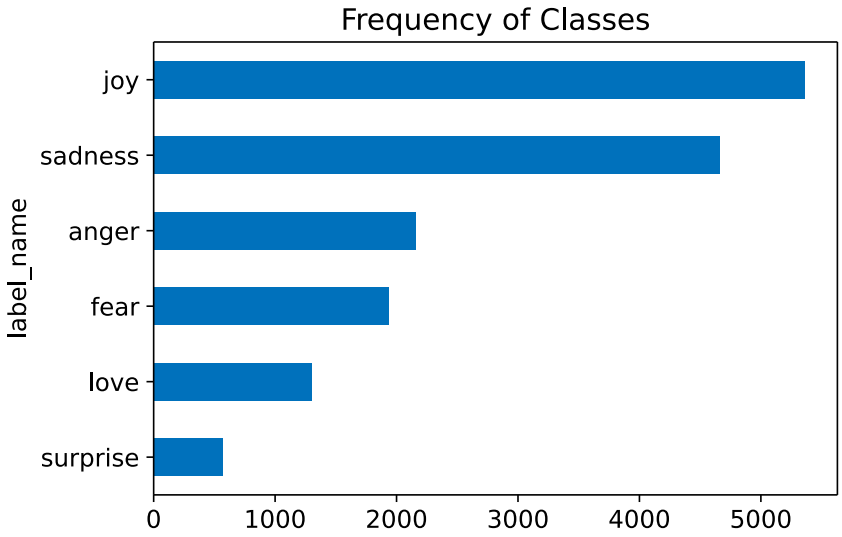

Figure 1: Class distribution of the CARER emotion training set. The dataset is imbalanced, with "joy" and "sadness" being significantly more frequent than "love" and "surprise."

grid search for the fine-tuning setup, analyse classification errors, and finally construct adversarial examples to probe the model's limitations.

2 Methods

2.1 Dataset

The CARER emotion dataset [Saravia et al., 2018][1] contains 20 000 English tweets, partitioned into 16 000 training, 2 000 validation, and 2 000 test samples. Each tweet is labelled with one of six emotion classes: sadness (0), joy (1), love (2), anger (3), fear (4), and surprise (5). The dataset is moderately imbalanced: "joy" and "sadness" are the most frequent categories, while "love" and "surprise" are the least frequent (Figure 1). Despite this imbalance, most tweets are short (around 15 words on average), well within DistilBERT's maximum sequence length of 512 tokens, so no truncation is necessary for the vast majority of samples.

2.2 Tokenization

DistilBERT uses the WordPiece subword tokenizer with a vocabulary size of 30 522. Unlike simple character-level or word-level tokenization, WordPiece can represent rare words as sequences of more frequent subword units, striking a balance between vocabulary size and sequence length. Special tokens are added to each input sequence: a [CLS] token at the beginning (whose final hidden state serves as the aggregate representation for classification) and a [SEP] token at the end. Padding tokens [PAD] are appended to shorter sequences so that all inputs in a batch have the same length; an attention mask tells the model to ignore these padded positions. As a pedagogical warm-up, we first implement a simple character-level token-to-index mapping before switching to the pre-trained WordPiece tokenizer for all downstream experiments.

2.3 DistilBERT as a Feature Extractor

DistilBERT [Sanh et al., 2019] is a distilled version of BERT with approximately 66 million parameters, roughly 40% smaller than the base BERT model while retaining about 97% of its language understanding performance. In the feature extraction paradigm, we freeze DistilBERT's pre-trained weights and use it solely as a fixed encoder. For each tweet, we extract the 768-dimensional hidden state corresponding to the [CLS] token—this vector serves as a dense representation of the entire

[1]The dataset is available at `https://github.com/dair-ai/emotion_dataset`.

Table 1: Validation accuracy of Scikit-Learn classifiers trained on frozen DistilBERT [CLS] features.

Classifier	Validation Accuracy
Softmax (Logistic Regression)	0.6345
SVM (RBF kernel)	0.6215
KNN ($k = 50$, cosine)	0.5275
Random Forest	0.5155
Dummy (most frequent)	0.3520

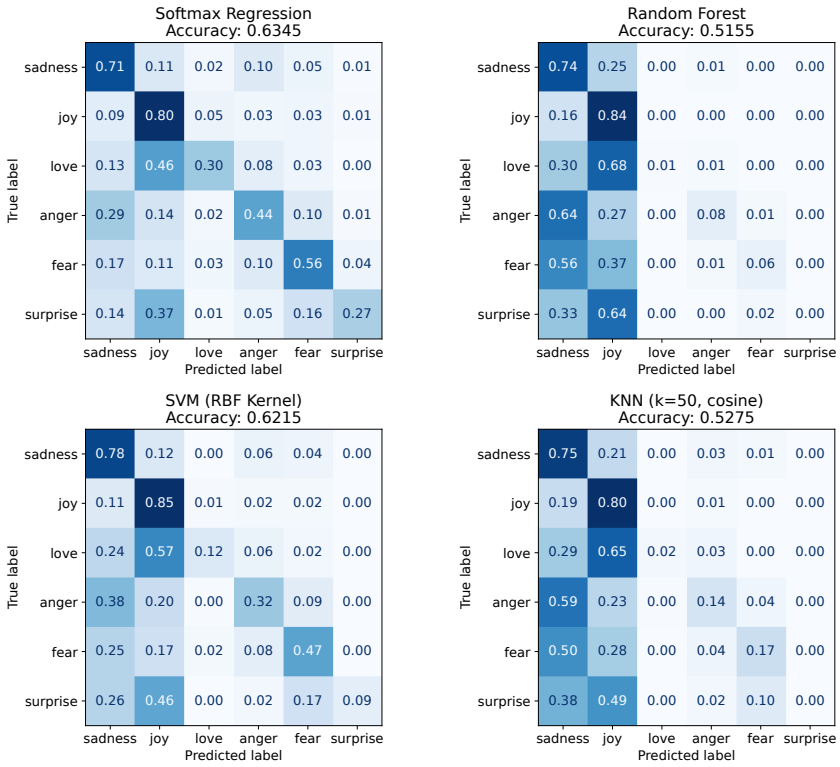

Figure 2: Normalised confusion matrices for the four classifiers on the validation set. Softmax and SVM produce qualitatively similar patterns, while Random Forest and KNN show substantially more off-diagonal confusion.

input sequence. The extraction is performed efficiently over the full dataset using HuggingFace's `Dataset.map()` with batched processing.

Once the [CLS] features are computed for all training and validation samples, we train several Scikit-Learn classifiers on top of them. Table 1 reports the validation accuracy for each model. The softmax regression model (implemented via `LogisticRegression` with the multinomial setting) achieves the highest accuracy at 63.45%, followed closely by the SVM with an RBF kernel at 62.15%. Random Forest and K-nearest neighbours perform markedly worse at 51.55% and 52.75%, respectively, suggesting that tree-based and distance-based methods struggle in the 768-dimensional feature space. All four models comfortably exceed the majority-class baseline of 35.2% established by a `DummyClassifier`.

2.4 Fine-tuning

In the fine-tuning approach, we initialise a `AutoModelForSequenceClassification` with a randomly initialised classification head on top of the pre-trained DistilBERT body, and train the entire model end-to-end using the HuggingFace `Trainer` API. The baseline configuration uses a learning rate of 1×10^{-5}, a batch size of 16, a weight decay of 0.1, and early stopping with patience 2 for

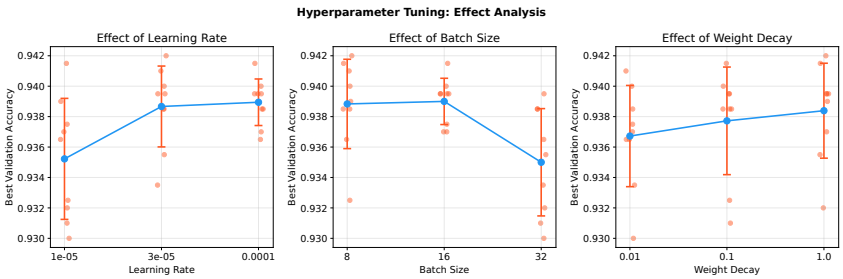

Figure 3: Marginal effect of each hyperparameter on best validation accuracy across the 27-experiment grid. Dots show individual experiment results; lines connect group means with ± 1 standard deviation error bars.

Table 2: Top-5 hyperparameter combinations by best validation accuracy from the $3 \times 3 \times 3$ grid search.

Learning Rate	Batch Size	Weight Decay	Val. Accuracy
3×10^{-5}	8	1.0	0.9420
1×10^{-5}	16	0.1	0.9415
1×10^{-4}	8	1.0	0.9415
3×10^{-5}	8	0.01	0.9410
1×10^{-4}	8	0.1	0.9400

a maximum of 5 epochs. This baseline achieves a validation accuracy of approximately 93.9%—a dramatic improvement of over 30 percentage points compared to the best feature extraction result, demonstrating that end-to-end adaptation of the pre-trained representations is essential for this task.

To further explore the hyperparameter space, we conduct a $3 \times 3 \times 3$ grid search over learning rate ($[1 \times 10^{-5}, 3 \times 10^{-5}, 1 \times 10^{-4}]$), batch size ($[8, 16, 32]$), and weight decay ($[0.01, 0.1, 1.0]$), yielding 27 experiments in total. Each experiment loads a fresh copy of the pre-trained model and is trained independently with early stopping. To accelerate the search, experiments are distributed across all available GPUs using Python's `ProcessPoolExecutor` with a spawn context for safe CUDA multiprocessing. Figure 3 shows the marginal effect of each hyperparameter on validation accuracy.

3 Experiments and Results

3.1 Feature Extraction Results

The feature extraction experiments confirm that linear classifiers (softmax and SVM) are better suited to the 768-dimensional [CLS] embedding space than non-linear alternatives. The softmax model's accuracy of 63.45% serves as the reference point for evaluating the gains from fine-tuning. The confusion matrices in Figure 2 show that all classifiers struggle most with the "surprise" and "love" categories, which are the least frequent classes in the training set.

3.2 Hyperparameter Tuning Results

The 27-experiment grid search reveals that the optimal configuration (learning rate 3×10^{-5}, batch size 8, weight decay 1.0) achieves a best validation accuracy of 94.20%, a modest improvement over the baseline (93.90%). Table 2 lists the top five configurations. The hyperparameter effect analysis (Figure 3) shows that learning rate has the largest marginal impact on performance, with moderate values (3×10^{-5}) slightly outperforming extremes. Weight decay provides a mild regularisation benefit, while batch size has a relatively small effect within the tested range. Overall, the baseline configuration is already near-optimal, suggesting that the fine-tuning process is robust to moderate hyperparameter variation.

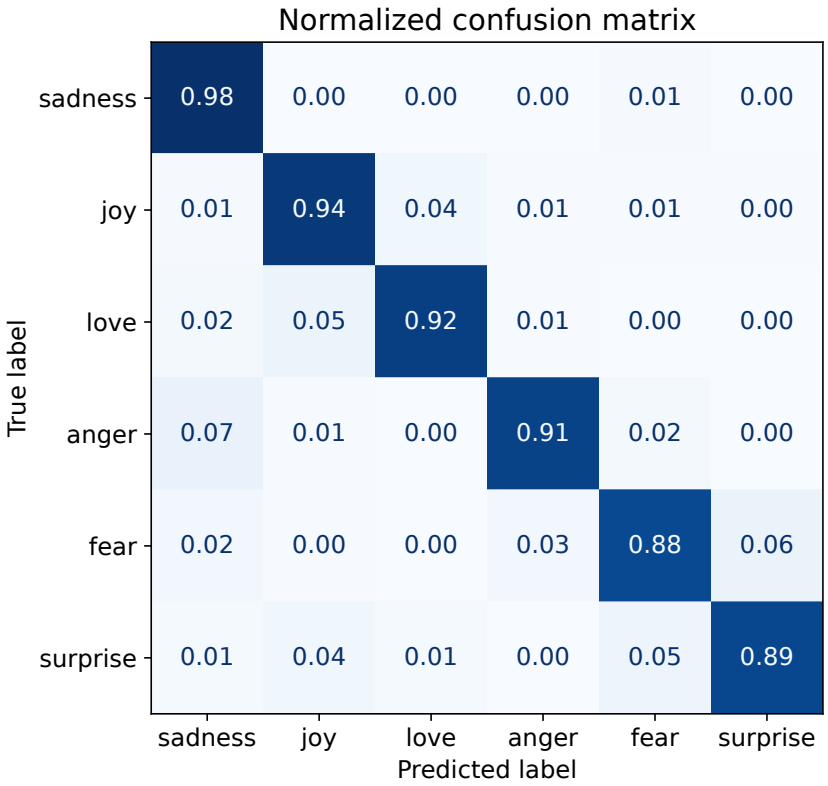

Figure 4: Normalised confusion matrix of the best fine-tuned model on the validation set. The diagonal entries are close to 1.0 for most classes, with the primary confusion occurring between "surprise" and "joy."

3.3 Error Analysis

To understand the model's failure modes, we sort the validation samples by their cross-entropy loss and examine the highest- and lowest-loss examples. The highest-loss samples predominantly belong to the "surprise" category but are confidently misclassified as "joy." This pattern can be attributed to two factors: (i) "surprise" and "joy" share a substantial amount of positive vocabulary (e.g., "amazing," "impressed," "delighted"), making their [CLS] representations difficult to separate; and (ii) the significant class imbalance biases the decision boundary toward the majority class "joy." In contrast, the lowest-loss samples are almost exclusively "joy" examples containing unambiguous positive keywords such as "delighted," "wonderful," and "pleased," confirming that the model's strongest performance is on the most frequent and lexically distinct class.

3.4 Adversarial Examples

We construct five adversarial examples using two complementary strategies. First, we manually craft two examples designed to exploit specific weaknesses of Transformer-based classifiers:

1. **Negation scrambling**: A sentence with multiple negations ("I'm not unhappy, not upset, and not at all angry") combined with clearly sad context ("terrible news"). While a human reader interprets the intended emotion as sadness, the model predicts "joy," demonstrating that multiple negation markers confuse the model into ignoring the underlying sentiment.
2. **Sarcasm / irony**: A short sarcastic remark ("Oh wonderful, just what I needed – two flat tyres and a burnt coffee.") The intended emotion is anger, but the model predicts "joy" with moderate confidence, indicating that it relies on surface-level positive words rather than pragmatic understanding.

Table 3: Summary of five adversarial examples. "Source / Target" indicates the intended emotion and the misclassification goal. "Prediction" is the model's output on the adversarial text.

Strategy	Source / Target	Prediction	Success
Negation scrambling	sadness $\rightarrow$ joy	joy	Yes
Sarcasm / irony	anger $\rightarrow$ joy	joy	Yes
HotFlip (gradient)	joy $\rightarrow$ love	love	Yes
HotFlip (gradient)	sadness $\rightarrow$ love	love	Yes
HotFlip (gradient)	anger $\rightarrow$ love	love	Yes

Second, we implement a HotFlip-style gradient-based attack that automatically searches for token replacements to push a correctly classified sample toward a target misclassification. From the validation set, we collect up to ten high-confidence (score > 0.9) correct predictions for each of three classes: "joy," "sadness," and "anger." For each class, we take the first available sample and attempt to push its prediction toward "love" using a HotFlip-style gradient attack. The algorithm computes the gradient of the target-class loss with respect to the input embeddings, scores candidate token replacements by their approximate effect on the loss, and iteratively applies the best replacement. Only complete word tokens (not "##" subword continuations) are considered. The attack runs for up to 15 steps with a maximum of 3 replacements; if one candidate fails, the next from the pool is tried. Results are summarised in Table 3.

4 Conclusion

This report presented an emotion recognition system for Twitter text based on DistilBERT. Our experiments demonstrate a substantial gap between feature extraction (63%) and fine-tuning (94%), confirming that end-to-end adaptation of pre-trained representations is critical for downstream text classification tasks. Error analysis reveals that class imbalance and vocabulary overlap between "surprise" and "joy" are the primary causes of misclassification. Adversarial examples constructed through both manual crafting and gradient-based attacks show that the model relies heavily on surface-level token patterns rather than deeper semantic or pragmatic understanding, leaving it vulnerable to negation scrambling and sarcasm.

Several limitations should be acknowledged. The experiments are limited to a single architecture (DistilBERT) and a single dataset (CARER emotion). The grid search, while covering three factors, uses a relatively coarse grid. Future work could extend this study by exploring larger pre-trained models (RoBERTa, DeBERTa), incorporating adversarial training to improve robustness, and investigating multi-task learning for joint emotion and sentiment analysis.

References

Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 4171–4186. Association for Computational Linguistics, 2019. doi: 10.18653/v1/N19-1423.

Victor Sanh, Lysandre Debut, Julien Chaumond, and Thomas Wolf. DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter. *arXiv preprint arXiv:1910.01108*, 2019.

Elvis Saravia, Hsien-Chi Tommy Liu, Yen-Hao Huang, Junlin Wu, and Yi-Shin Chen. CARER: Contextualized affect representations for emotion recognition. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, pages 3687–3697. Association for Computational Linguistics, 2018. doi: 10.18653/v1/D18-1404.

Lewis Tunstall, Leandro von Werra, and Thomas Wolf. *Natural Language Processing with Transformers: Building Language Applications with Hugging Face*. O'Reilly Media, Sebastopol, CA, revised edition, 2022. ISBN 9781098136796.